

Application Registration

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Enabling It	2
1.1. Why the environment is configured, not discovered	2
2. Implementing a Registrar	3
2.1. Startup must be a one-time hit	3
2.2. Two rules that are easy to get wrong	4
3. What the Environment Answers	5
4. When It Runs	6
5. Failure Policy	7
6. Registration Is Not Configuration	8
7. See Also	9

A service that has started is not an environment that has been provisioned. Liveness probes report that a process is running; nothing about a green container says the realm it reads was ever migrated, or that its seeds were ever applied. When they were not, the symptoms surface deep in the request path—an unresolvable subject, a missing functional domain, a 403 on an annotated route—and each reads as a separate defect rather than the one cause they share: the application was never installed into this environment.

Application registration closes that gap. An application announces itself to its environment at startup, and the environment answers with what it knows—installing, seeding, and migrating on first arrival, or naming precisely what remains.

This is an **optional** capability. The framework defines the contract and never the control plane: with registration disabled, which is the default, nothing here runs and the framework starts exactly as it does without it.

Chapter 1. Enabling It

Registration is enabled by configuration and satisfied by an implementation on the classpath.

```
quantum.application-registration.enabled=true
quantum.application-registration.provider=system-manager
quantum.application-registration.application-id=my-app
quantum.application-registration.environment=production
quantum.application-registration.realms=my-app-P1,my-app-P2
```

`provider` selects the implementation. It matches either the registrar's `getName()` or its fully-qualified class name, so both of the following are valid:

```
quantum.application-registration.provider=system-manager
quantum.application-
registration.provider=com.e2eq.system.client.registration.SystemManagerRegistrar
```

1.1. Why the environment is configured, not discovered

`application-id` and `environment` are required and are never inferred. Discovery is tempting and wrong: the thing that would answer "which environment am I in" is the registry we are trying to reach, and a service that guesses can register into the **wrong** environment. That is worse than failing to start — the wrong environment's directory then claims a service that is not there, and the right one never learns about it.

For the same reason, the address of the control plane is a configured seed. The chicken-and-egg is broken by configuration, deliberately, at exactly one point.

Chapter 2. Implementing a Registrar

Implement `ApplicationRegistrar` and make it a CDI bean.

```
public interface ApplicationRegistrar {  
    String getName();  
    boolean isRegistered(RegistrationContext context);  
    RegistrationOutcome register(RegistrationContext context) throws  
    RegistrationException;  
}
```

`ApplicationRegistrarFactory` discovers implementations through CDI and resolves the configured one, mirroring how `AuthProviderFactory` resolves authentication providers. There is deliberately one way to plug an implementation into the framework rather than two.

2.1. Startup must be a one-time hit

Registration runs on every start but must only be **paid for** once.

`isRegistered(context)` is the cheap path and is called first on every boot. `register(context)` runs only when it answers `false`.



`isRegistered` **must not perform a remote call**. It runs on the path that decides whether the expensive work is needed at all. An implementation that asks the control plane "have I registered?" on every boot turns a one-time install into a per-restart round trip — and a crash-looping service into a load generator against the control plane.

Answer it from a locally persisted receipt keyed by `RegistrationContext.getFingerprint()`: a single indexed local read.

The fingerprint covers everything whose change should force re-registration:

Component	Why it is included
<code>applicationId</code>	A different application is a different registration.
<code>applicationVersion</code>	A new build may declare new realms, new contracts, or new seeds.
<code>environmentRef</code>	The same build in a different environment has registered nowhere yet.
<code>requiredRealms</code>	A newly required realm may be unmigrated even though the application is known.

The same build redeployed into the same environment produces the same fingerprint and costs one local read. A version bump or a newly required realm produces a different one and registers again.

2.2. Two rules that are easy to get wrong

Return `false` from `isRegistered` when unsure. A redundant registration is idempotent and cheap to absorb; a skipped one leaves an application running against an environment that was never prepared for it.

Record the receipt only on **fully successful** registration. Writing one for a partial install makes the next boot skip work that never finished.

Chapter 3. What the Environment Answers

`register` returns a `RegistrationOutcome`:

Status	Meaning
REGISTERED	Registered during this call.
ALREADY_REGISTERED	Already registered; nothing was written.
NOT_READY	The environment is reachable but not ready to serve this application.
REFUSED	The environment refused this registration outright.

`NOT_READY` carries `getRemainingActions()` — ordered, human-actionable steps. This is the part that repays the protocol. A service that cannot be served by its environment says which step is missing, instead of starting anyway and failing later inside an unrelated request.

Chapter 4. When It Runs

Registration runs **after** startup completes, on a background thread, not inside the startup sequence.

This is not an optimisation. Registering can cause the environment to call back into the very service that is registering — installing an application provisions its realm by asking that application’s own tenant plane to do it. Startup observers run before the HTTP server accepts connections, so registering inline deadlocks a service against itself: it waits for a callback it cannot yet answer.

The service therefore becomes **live** first and registers immediately afterwards. This is the same reason registration belongs in front of readiness rather than liveness: a service whose environment is not provisioned is running, and should not be serving traffic.

Chapter 5. Failure Policy

```
quantum.application-registration.fail-on-error=true
```

Defaults to **true**, which shuts the service down.

Because registration runs off the startup thread, throwing would stop nothing—the exception would be swallowed by a background worker and the service would go on serving traffic against an environment that does not know it. Shutting down is what "fail on error" has to mean once registration is asynchronous.

An application whose environment does not know it is an application whose realms may be unmigrated and whose seeds may be absent; starting anyway converts one clear startup error into a stream of unrelated-looking failures later.

Set it to **false** where that trade is wrong—a developer machine with no control plane running, for instance. Whether a failed registration should stop the service is a property of the deployment, not of the code that failed, which is why it is configuration and not a decision the registrar makes.

Chapter 6. Registration Is Not Configuration

Registration answers one question: **did this application land in this environment?** That is infrastructure — the realm’s datastore, its migrations, its seeds.

It deliberately does not answer **is this application fully configured?** An application missing, say, an API key is installed; it is not fully configured. Those are different problems, with different owners and different remedies.

Feature-level configuration gaps are reported through scoped action enablement (**quantum-action-enablement-quarkus**), which names which actions are unavailable and what is blocking each one. A blocker carries a **severity: error** withholds the feature, **warn** is advisory. Deciding what a given severity gates is the consuming control plane’s call, not the framework’s.

Chapter 7. See Also

- [Multiple Authentication Providers](#) — the same SPI-plus-configuration pattern for identity
- [Control Plane Architecture](#)